

CS 305: Computer Networks

Fall 2025

Lecture 8 + 9: Transport Layer

Ming Tang

Department of Computer Science and Engineering
Southern University of Science and Technology (SUSTech)

Chapter 3 outline

3.1 transport-layer services

3.2 multiplexing and demultiplexing

3.3 connectionless transport: UDP

3.4 principles of reliable data transfer

3.5 **connection-oriented transport: TCP**

- segment structure, RTT measurement
- reliable data transfer
- flow control
- **connection management**

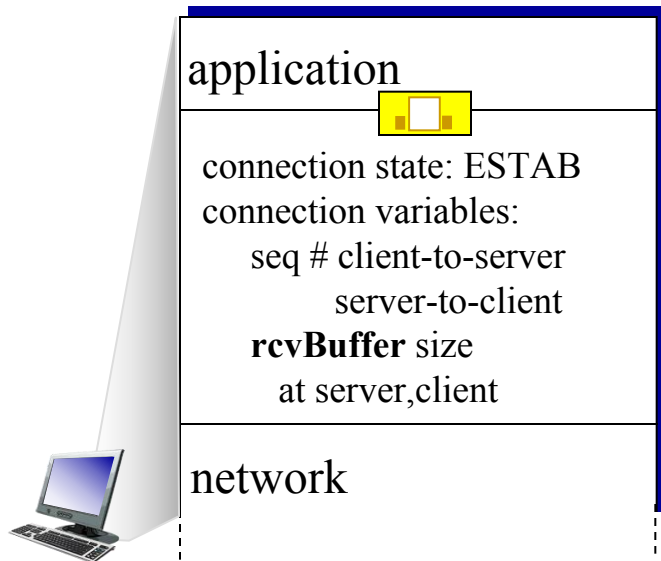
3.6 principles of congestion control

3.7 TCP congestion control

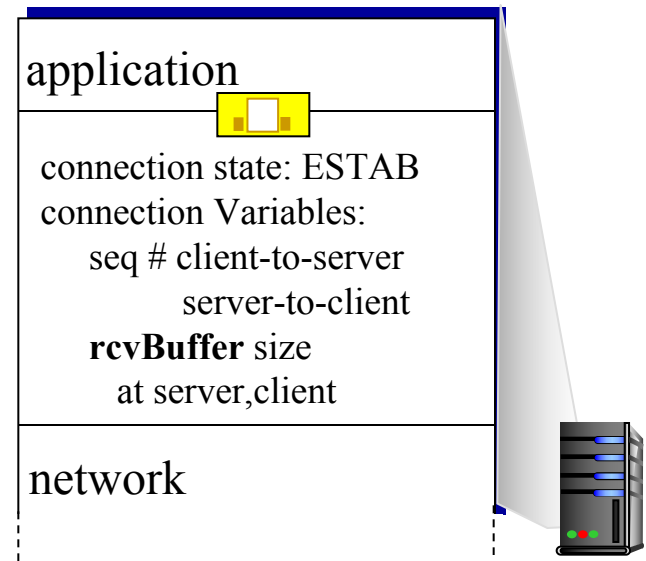
Connection Management

before exchanging data, sender/receiver “handshake” :

- ❖ agree to establish connection (each knowing the other willing to establish connection)
- ❖ agree on connection parameters



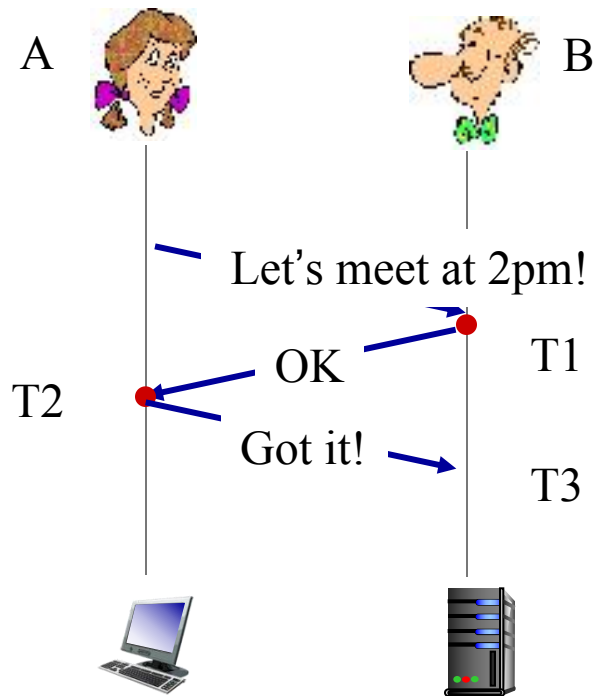
```
clientSocket = socket(AF_INET, SOCK_STREAM);  
clientSocket.connect((hostname,port number));
```



```
connectionSocket = welcomeSocket.accept();
```

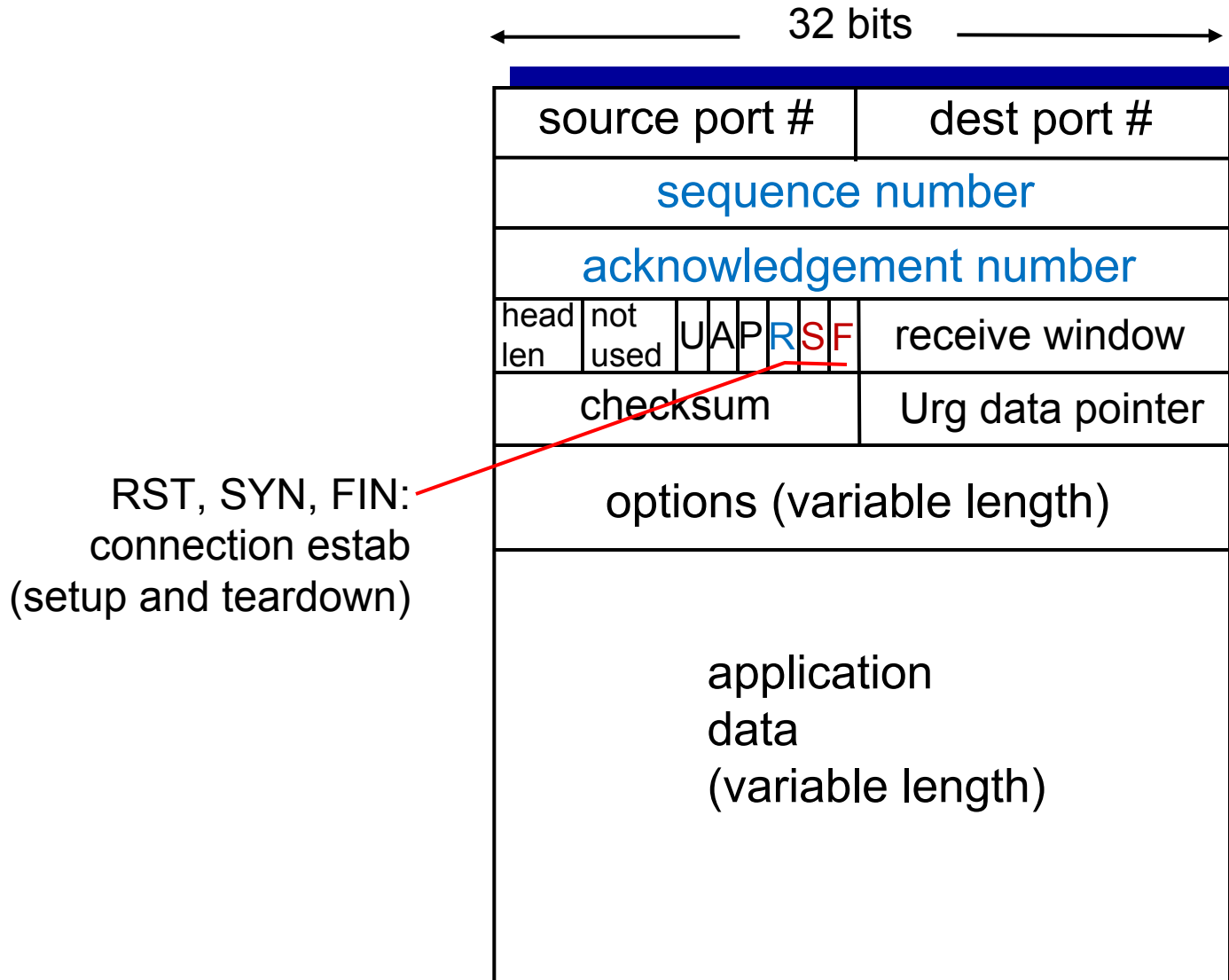
Agreeing to establish a connection

3-way handshake:



- ❖ T1: B knows A's transmitter and B's receiver is OK
- ❖ T2: A knows A's transceiver and B's transceiver is OK, B has no more information than T1
- ❖ T3: Both A and B know their transceiver are OK, they can start the communication!

TCP segment structure



TCP 3-way handshake

client state

LISTEN

SYNSENT

ESTAB

choose init seq num, x
send TCP SYN msg



SYNbit=1, Seq=x

SYNbit=1, Seq=y
ACKbit=1; ACKnum=x+1

received SYNACK(x)
indicates server is live;
send ACK for SYNACK;
this segment may contain
client-to-server data

SYNbit=0
ACKbit=1, ACKnum=y+1



choose init seq num, y
send TCP SYNACK
msg, acking SYN

received ACK(y)
indicates client is live

server state

LISTEN

SYN RCVD

ESTAB

Once these three steps have been completed, the client and server hosts can send segments containing data to each other.

- In each of these future segments, SYNbit=0

TCP: closing a connection

client state

ESTAB

clientSocket.close()

FIN_WAIT_1

can no longer
send but can
receive data

FIN_WAIT_2

wait for server
close

TIMED_WAIT

timed wait

CLOSED



FINbit=1, seq=x

ACKbit=1; ACKnum=x+1

FINbit=1, seq=y

ACKbit=1; ACKnum=y+1

can still
send data

can no longer
send data

server state

ESTAB

CLOSE_WAIT

LAST_ACK

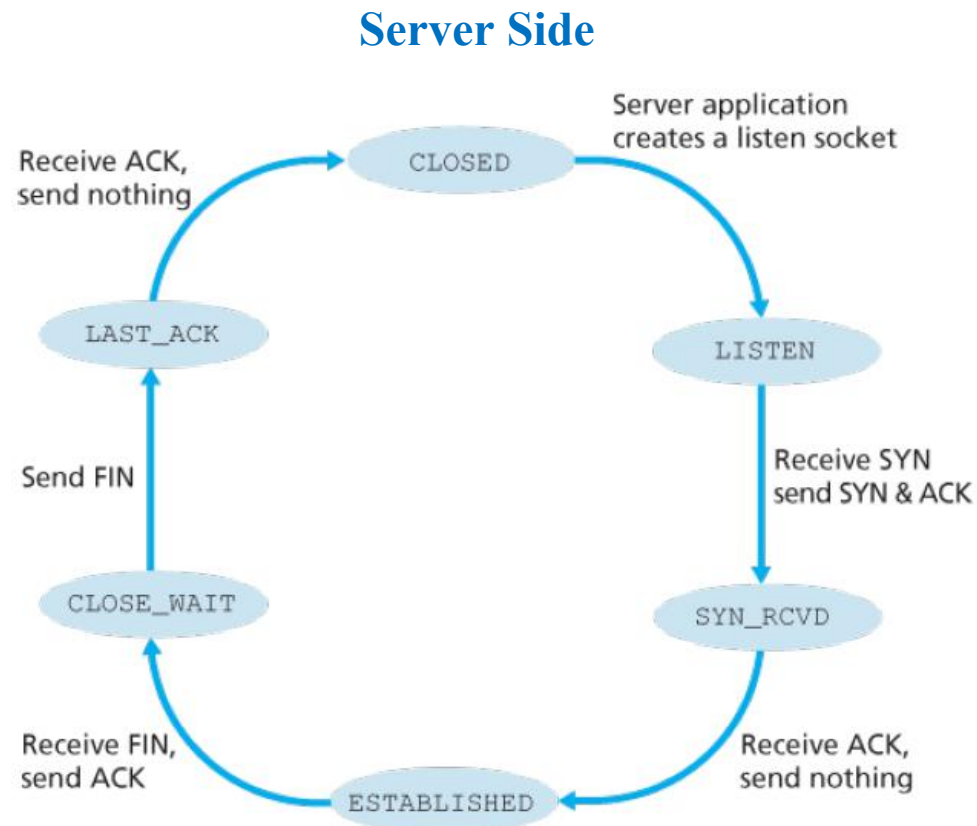
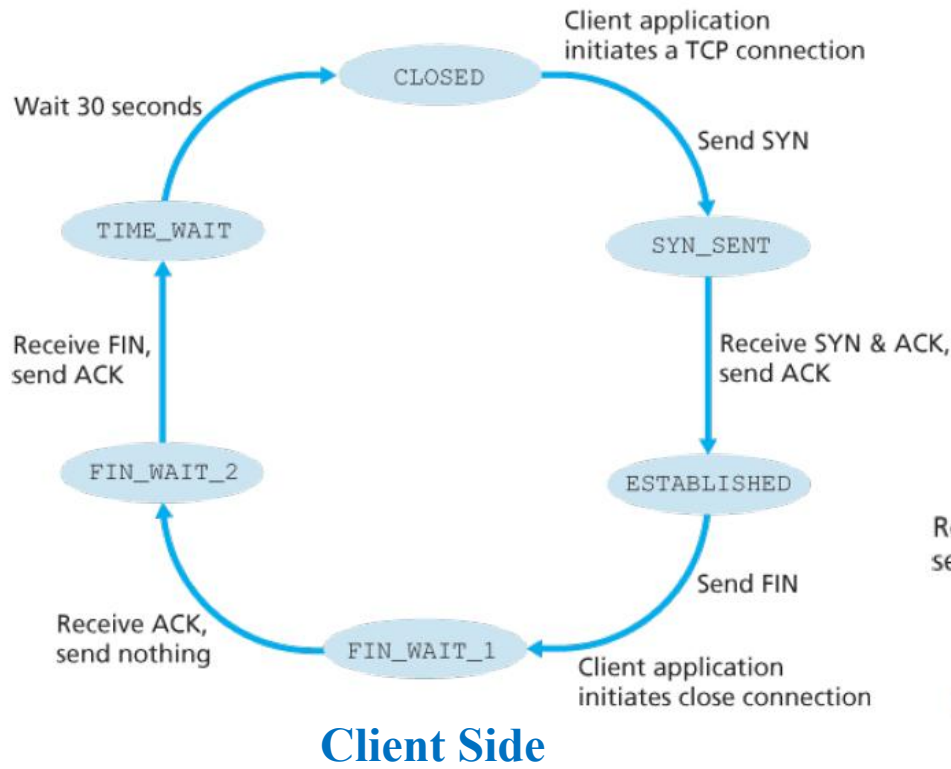
CLOSED

The TIME_WAIT state lets the TCP client resend the final acknowledgment in case the ACK is lost.

TCP: closing a connection

- ❖ Four-way handshaking
 - Either of the two processes participating in a TCP connection can end the connection.
- ❖ client, server each close their side of connection
 - send TCP segment with FIN bit = 1
- ❖ respond to received FIN with ACK
- ❖ Why FIN and ACK can not be sent in one msg as SYNACK in connection establishment?
 - The other side may still have packets need to be sent. It can not send FIN until the transmission is finished.

TCP States



Reset Segment

When a host receives a TCP segment whose port numbers or source IP address do not match with any of the ongoing sockets.

- ❖ Then the host will send a special reset segment to the source. RST flag bit is set to 1.
- ❖ “I don’t have a socket for that segment. Please do not resend the segment.”

Chapter 3 outline

3.1 transport-layer services

3.2 multiplexing and demultiplexing

3.3 connectionless transport: UDP

3.4 principles of reliable data transfer

3.5 connection-oriented transport: TCP

- segment structure
- reliable data transfer
- flow control
- connection management

3.6 principles of congestion control

3.7 TCP congestion control

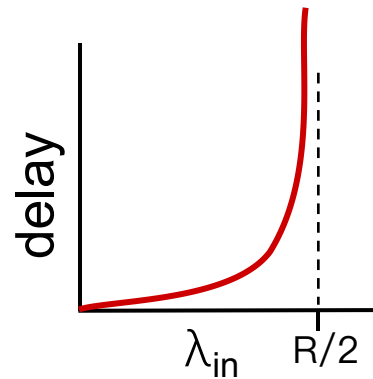
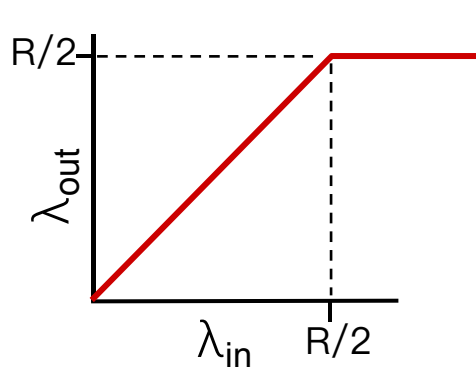
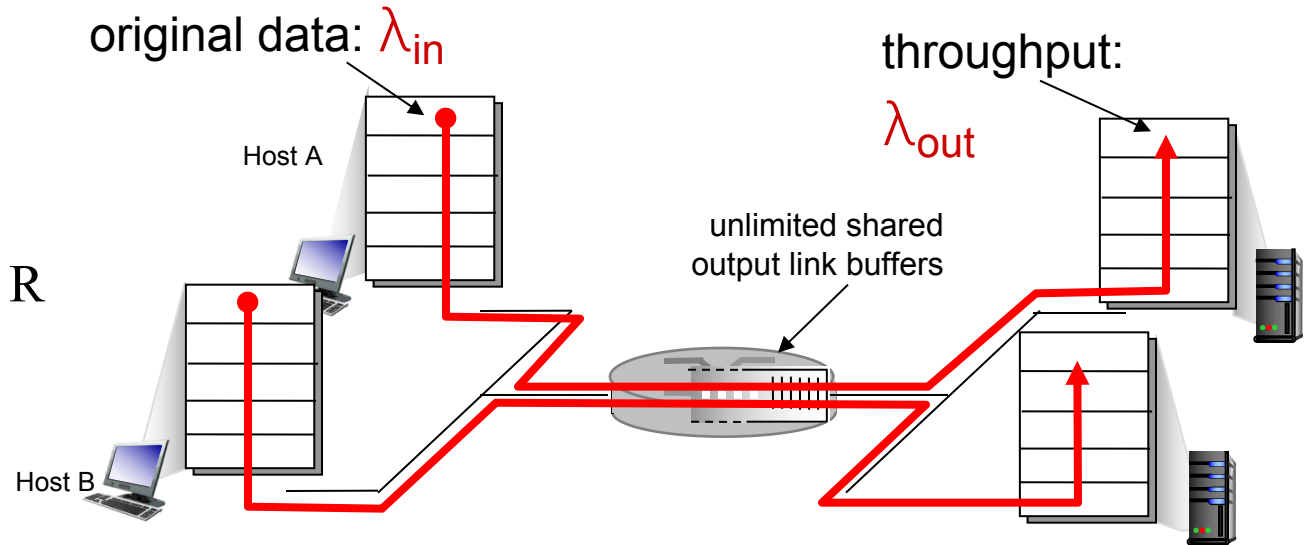
Principles of congestion control

Congestion:

- ❖ informally: “too many sources sending too much data too fast for *network* to handle”
- ❖ different from flow control!
- ❖ manifestations:
 - lost packets (buffer overflow at routers)
 - long delays (queueing in router buffers)
- ❖ a top-10 problem!

Causes/costs of congestion: scenario 1

- ❖ two senders, two receivers
- ❖ one router, infinite buffers
- ❖ output link capacity: R
- ❖ no retransmission

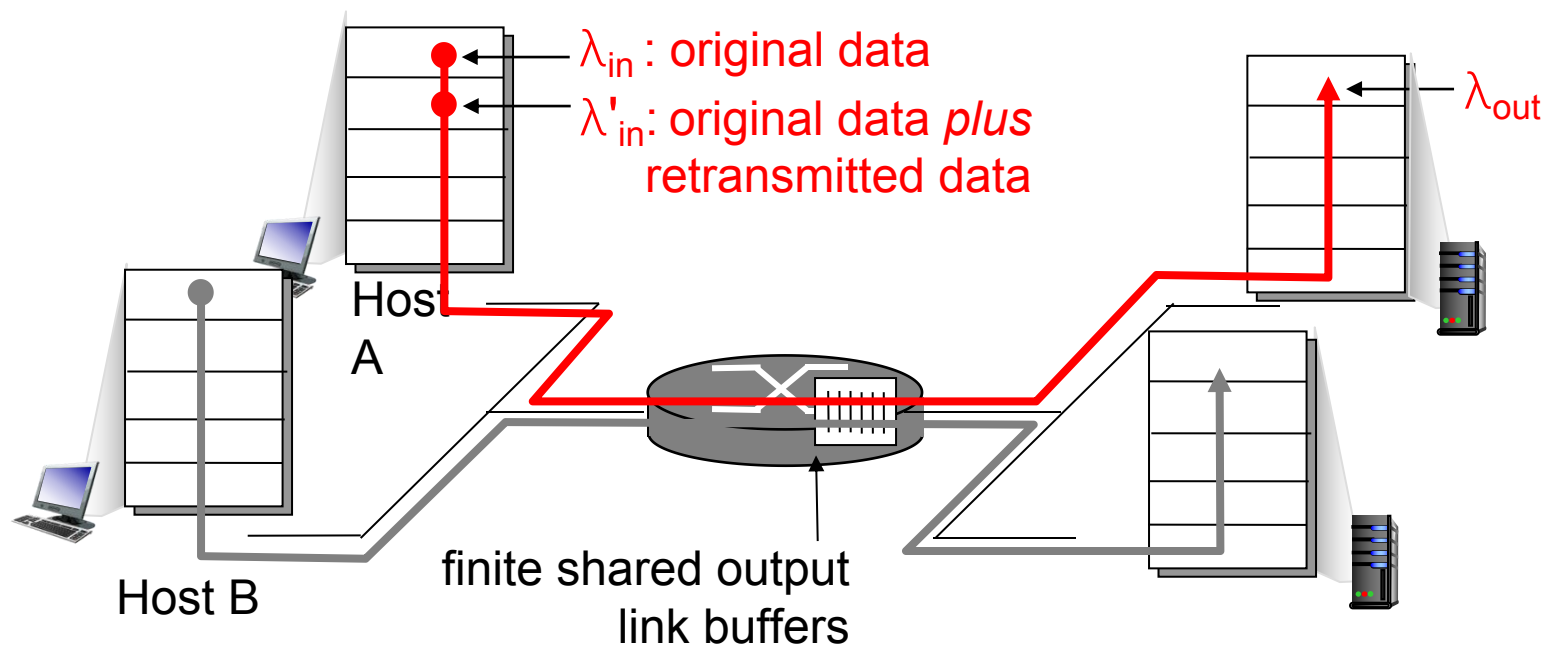


large queuing delays are experienced as the packet-arrival rate nears the link capacity.

- ❖ maximum per-connection throughput: $R/2$
- ❖ large delays as arrival rate, λ_{in} , approaches capacity

Causes/costs of congestion: scenario 2

- ❖ one router, *finite* buffers
- ❖ sender retransmission of timed-out packet
 - application-layer input = application-layer output: $\lambda_{in} = \lambda_{out}$
 - transport-layer input includes *retransmissions* : $\lambda'_{in} \geq \lambda_{in}$



Causes/costs of congestion: scenario 2

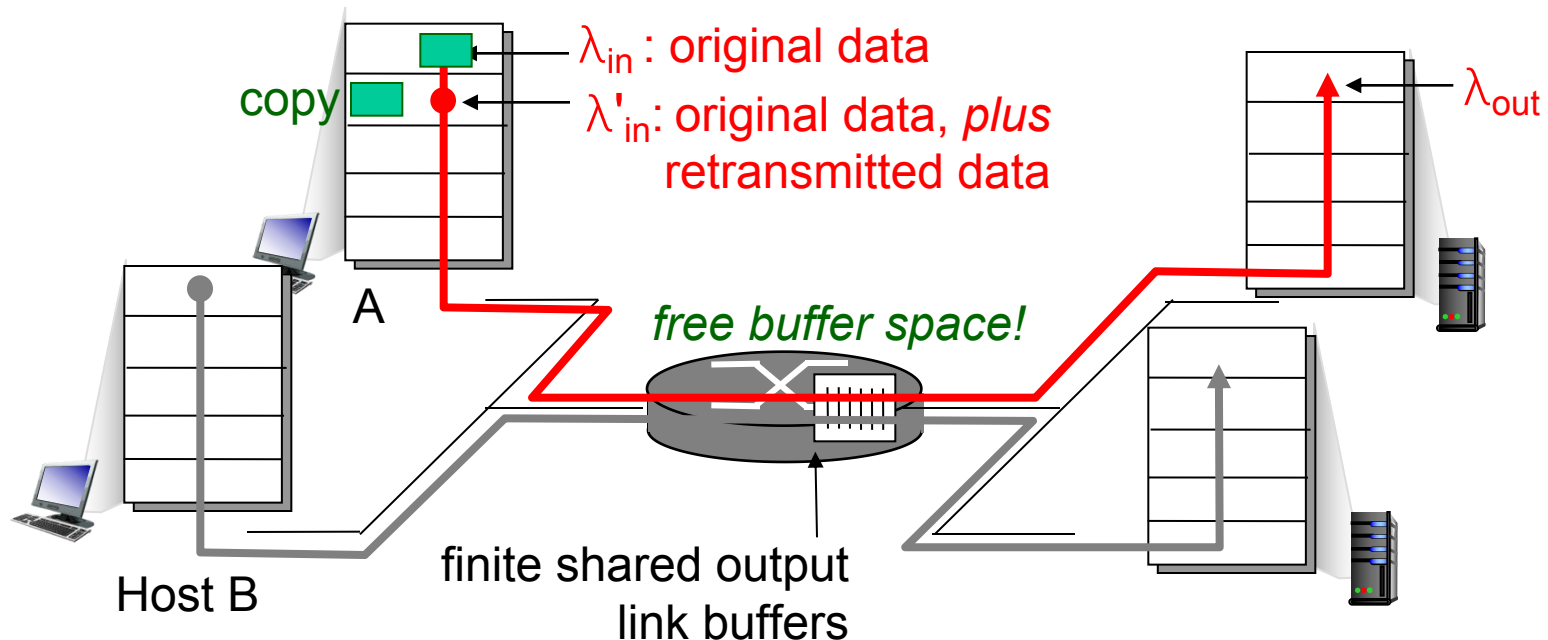
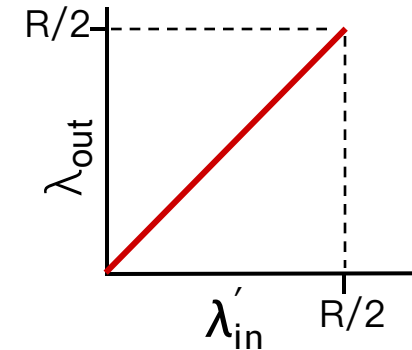
Idealization: perfect knowledge

- ❖ Sender sends only when router buffers available

发送端只在路由器有空缓冲时才发送

- ❖ No loss occurs: $\lambda'_{in} = \lambda_{in}$

不丢包，所以没有重传，传输层注入速率=原始数据速率



Causes/costs of congestion: scenario 2

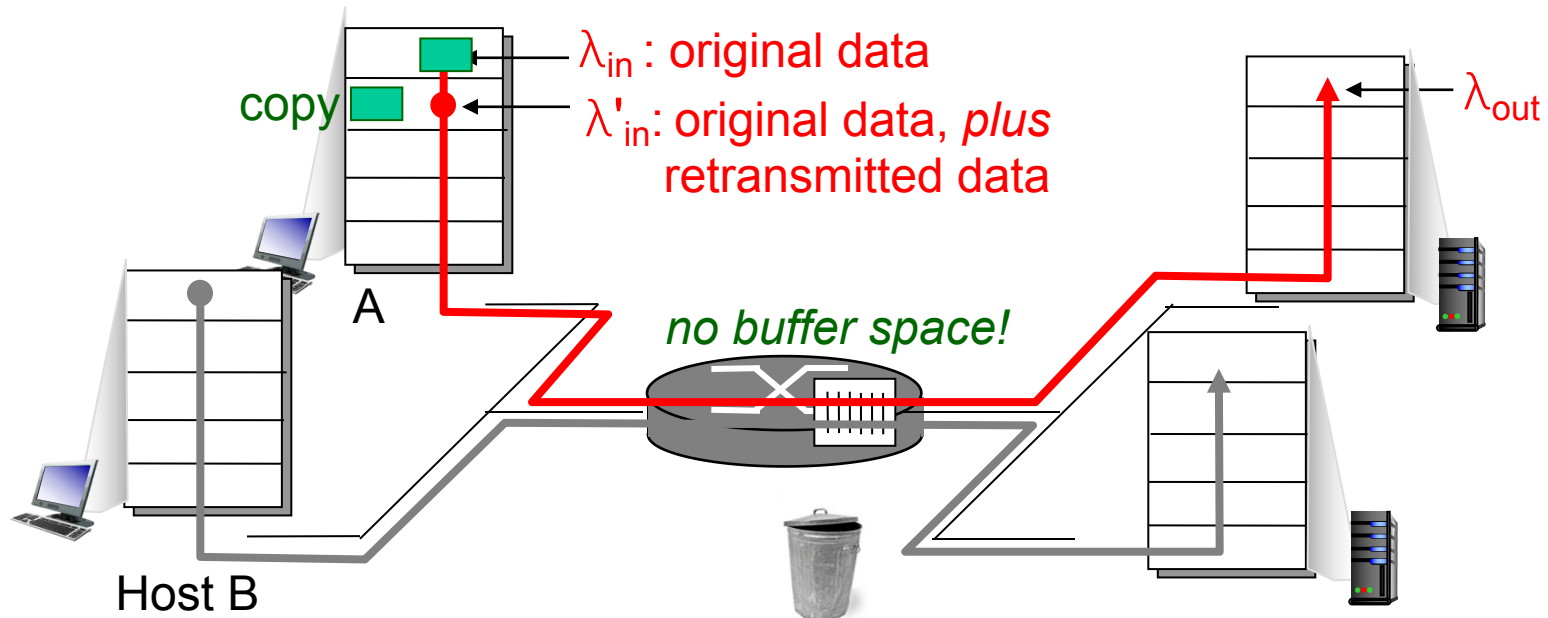
Idealization: **known loss** packets can be lost, dropped at router due to full buffers

发送端只有在“明确知道”某个包丢了才重传

❖ sender only resends if packet *known* to be lost

❖ $\lambda'_{in} \geq \lambda_{in}$

已知丢包场景中，数据包丢失发生在路由器缓冲区已满时，发送方只有在明确知道某个数据包丢失时才会重新发送。



Causes/costs of congestion: scenario 2

Idealization: known loss packets can be lost, dropped at router due to full buffers

❖ sender only resends if packet *known* to be lost

❖ $\lambda'_{in} \geq \lambda_{in}$

一开始（负载低）几乎不丢包：

$\lambda_{out} \approx \lambda_{in}$ （接近直线）

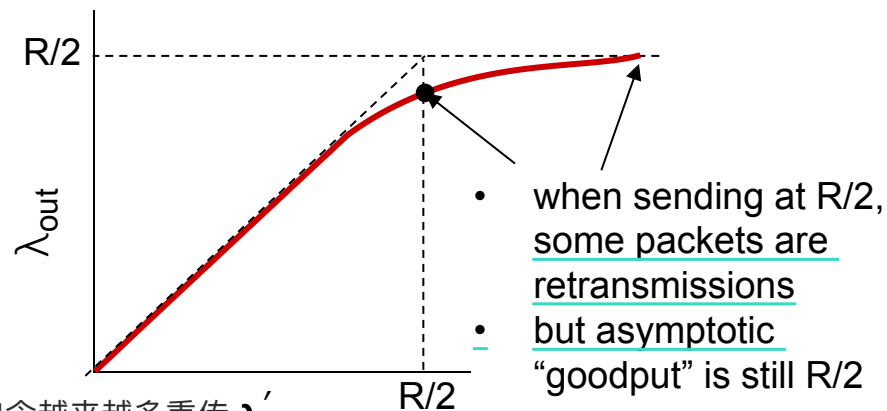
接近瓶颈后开始丢包：

你注入网络的总速率 λ'_{in} 会包含越来越多重传 λ_{in}
但瓶颈链路最多就 R （每条流最多 $R/2$ ）的服务能力

所以你会看到：

• 横轴（ λ'_{in} ）继续变大（因为重传越来越多）

• 纵轴（ λ_{out} ）有效吞吐）会增长到瓶颈吞吐后就不再



但长期平均下，能交付的“有效数据”（goodput）仍然卡在 $R/2$ 左右。

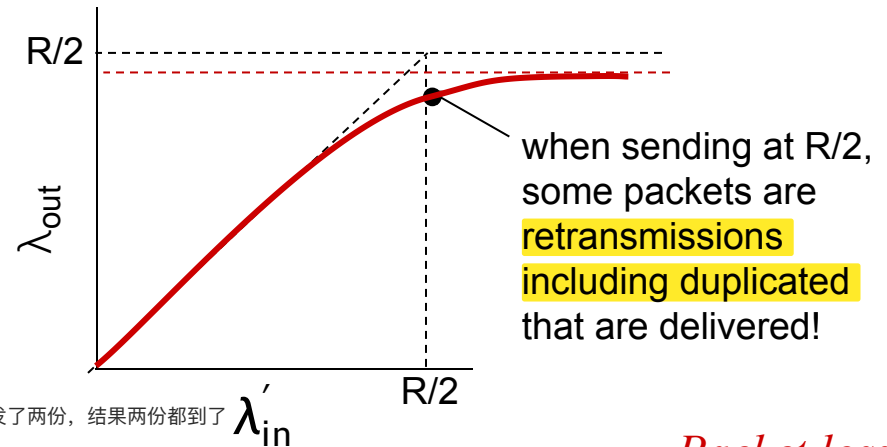
再加大发送，只会让重传比例升高，而有效吞吐不会明显更高

Causes/costs of congestion: scenario 2

Realistic: duplicates

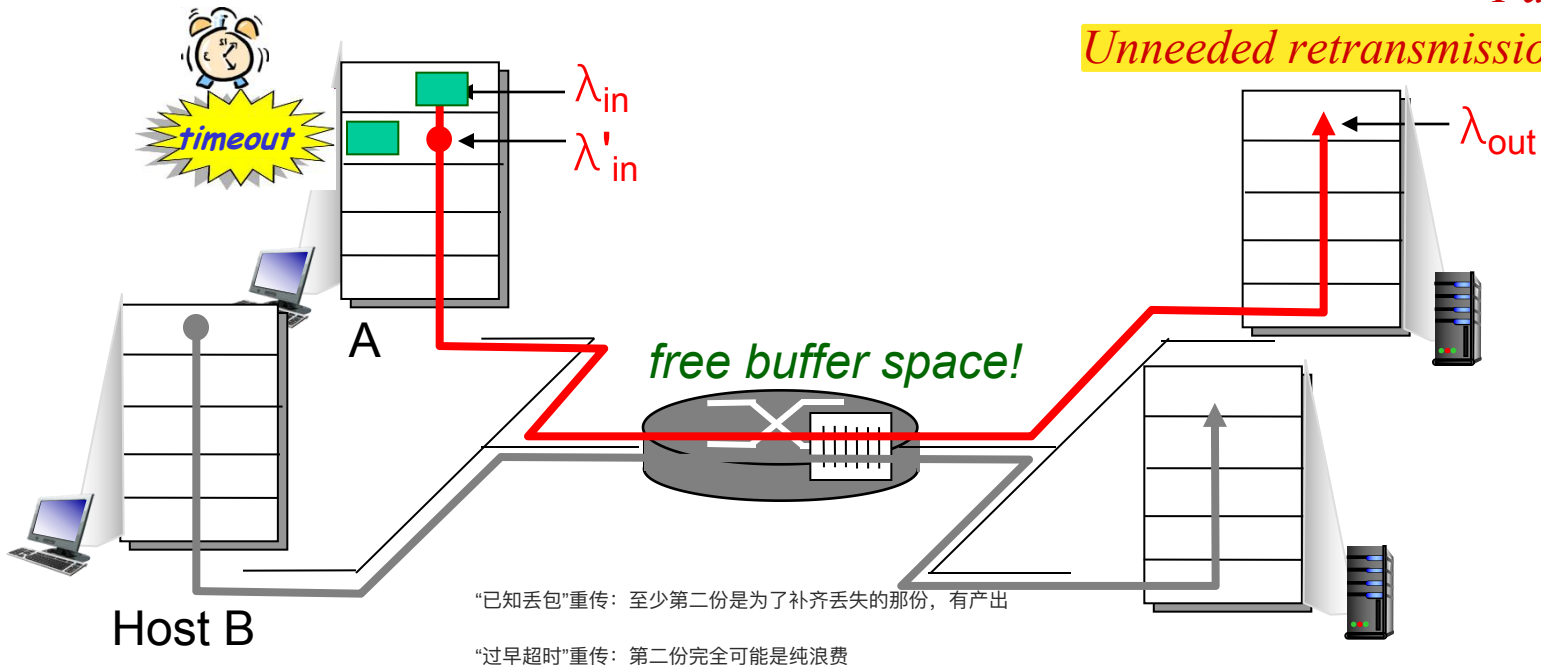
- ❖ packets can be lost, dropped at router due to full buffers
- ❖ sender times out prematurely, sending *two* copies, both of which are delivered

发送端过早超时，把同一个包发了两份，结果两份都到了



Packet loss

Unneeded retransmission: waste



For small values of λ_{in}^- :

- $$\lambda'_{in} = \lambda_{in}.$$

-
- Diagram illustrating a network topology for a multi-hop path with feedback. The network consists of four hosts (A, B, C, D) and three intermediate nodes. Host A is at the top left, Host B at the top right, Host C at the bottom right, and Host D at the bottom left. The network is connected via a central mesh of three nodes. Host A sends data (red line) to Host B. Host B sends data (green line) to Host C. Host C sends data (blue line) to Host D. Host D sends data (magenta line) back to Host A. The diagram illustrates a multi-hop path with feedback. The network is composed of four hosts (A, B, C, D) and three intermediate nodes. Host A is at the top left, Host B at the top right, Host C at the bottom right, and Host D at the bottom left. The network is connected via a central mesh of three nodes. Host A sends data (red line) to Host B. Host B sends data (green line) to Host C. Host C sends data (blue line) to Host D. Host D sends data (magenta line) back to Host A. The diagram illustrates a multi-hop path with feedback.
- Key components and labels:
- Host A**: Top left host.
 - Host B**: Top right host.
 - Host C**: Bottom right host.
 - Host D**: Bottom left host.
 - finite shared output link buffers**: Located at the intermediate nodes.
 - λ_{in} : original data**: Red line.
 - λ'_{in} : original data, *plus* retransmitted data**: Magenta line.
 - λ_{out}** : Green line.

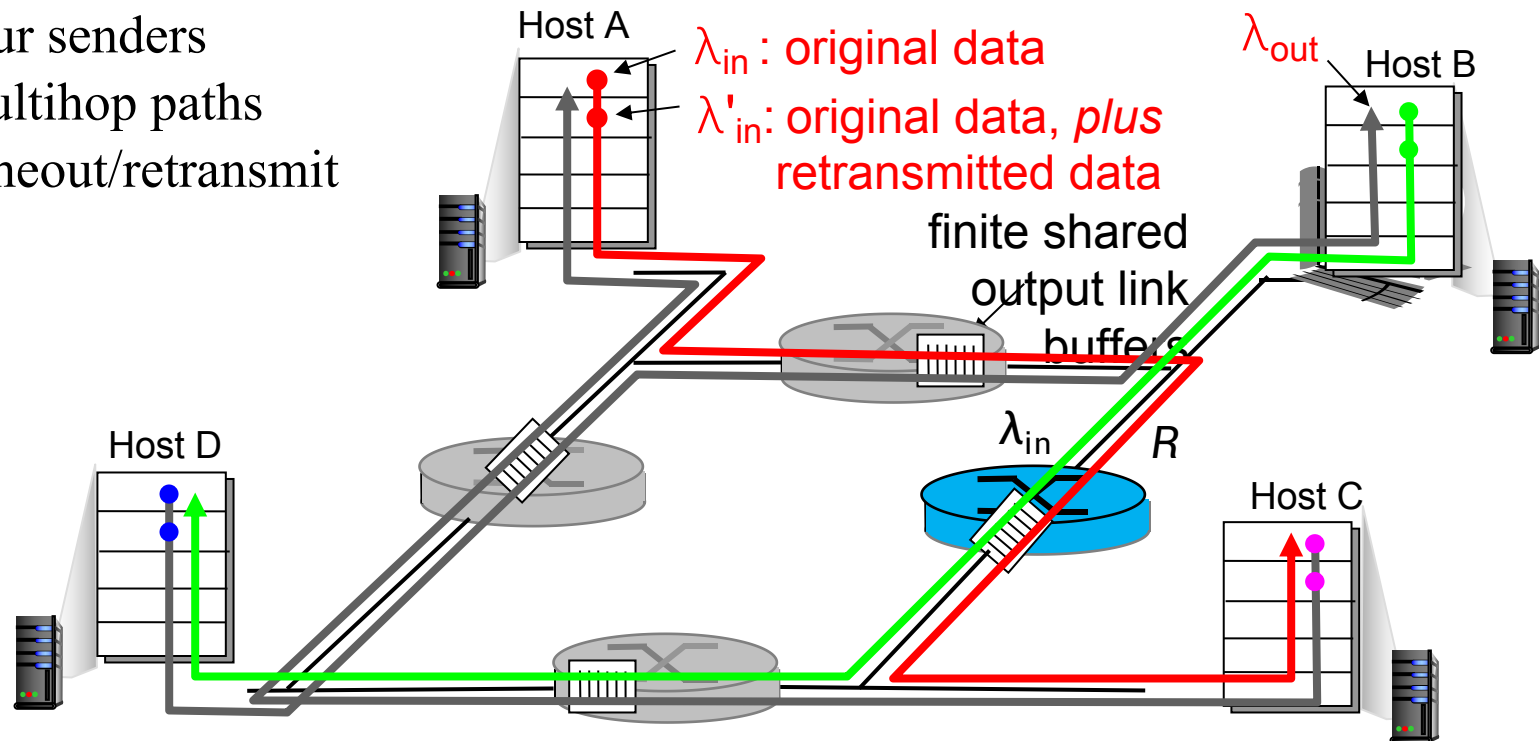
Causes/costs of congestion: scenario 3

Q: what happens as λ_{in} increases ?

A: as green λ_{in} increases, all arriving red pkts at upper queue are dropped, red throughput goes 0 关键机制：谁“更猛”，谁就占满队列

在多跳网络里，拥塞导致的丢包/重传，会把“前面几跳已经消耗的资源”也一起浪费掉

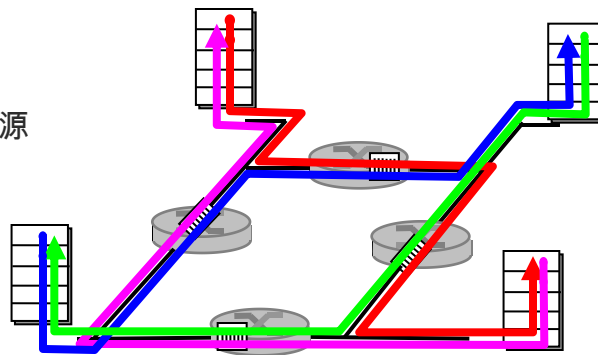
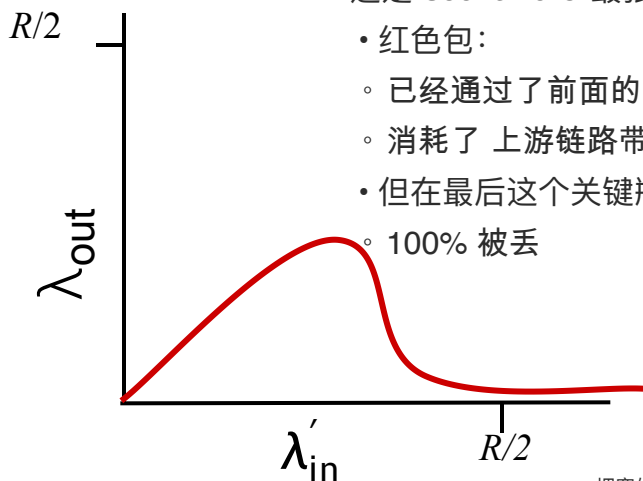
- ❖ four senders
- ❖ multihop paths
- ❖ timeout/retransmit



Causes/costs of congestion: scenario 3

这是 scenario 3 最狠的一点：

- 红色包：
 - 已经通过了前面的多个路由器
 - 消耗了 上游链路带宽 + 排队 + 转发资源
 - 但在最后这个关键瓶颈：
 - 100% 被丢



拥塞的另一个成本是，当数据包丢失时，任何“上游”传输所使用的带宽都会被浪费。具体来说，丢失的数据包虽然已经占用了网络资源进行传输，但最终并没有成功到达接收方，造成了带宽的浪费。

这种浪费不仅影响吞吐量（数据流量），还意味着网络中已经被占用的传输资源没有得到有效利用

another “cost” of congestion:

- ❖ when packet dropped, any “upstream” transmission capacity used for that packet was wasted!

Cause and Cost of Congestion

Cause

- Shared link; limited link capacity
- Sending at a high rate

Cost of Congestion

- Delay
- Packet lost and retransmission
- Unneeded retransmission: waste
- “upstream” transmission capacity was wasted

Approaches to Congestion Control

End-to-end congestion control:

发送端自己判断

- TCP segment loss or round-trip segment delay
- TCP decreases its **window size** accordingly

TCP不直接控制速率，控制的是拥塞窗口，实际发送速率
 $= \text{cwnd} / \text{RTT}$

Network-assisted congestion control:

路由器告诉发送端/接收端

- **routers** **provide** **feedback** to the sender and/or receiver
- **a single bit** indicating congestion at a link; the maximum host sending rate the router can support

1 bit 拥塞标志

显式速率反馈

Chapter 3 outline

3.1 transport-layer services

3.2 multiplexing and demultiplexing

3.3 connectionless transport: UDP

3.4 principles of reliable data transfer

3.5 connection-oriented transport: TCP

- segment structure
- reliable data transfer
- flow control
- connection management

3.6 principles of congestion control

3.7 TCP congestion control

TCP congestion control: additive increase multiplicative decrease

- TCP use end-to-end congestion control
- have each sender limit the rate at which it sends traffic into its connection as a function of perceived network congestion

TCP 用 端到端拥塞控制 (end-to-end) : 路由器不直接告诉你, 你靠丢包/延迟自己推断

每个发送端根据“感知到的拥塞程度”主动限制自己的发送速率

Questions for achieving congestion control:

Q1: How does a TCP sender limit the rate at which it sends traffic into its connection?

Q2: How does a TCP sender perceive that there is congestion on the path between itself and the destination?

Q3: What algorithm should the sender use to change its send rate as a function of perceived end-to-end congestion?

TCP congestion control: additive increase multiplicative decrease

Questions for achieving congestion control:

Q1: How does a TCP sender limit the rate at which it sends traffic into its connection?

Congestion window: **cwnd**

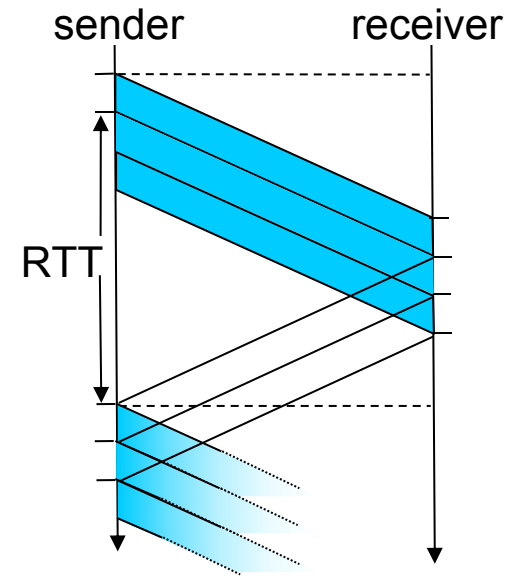
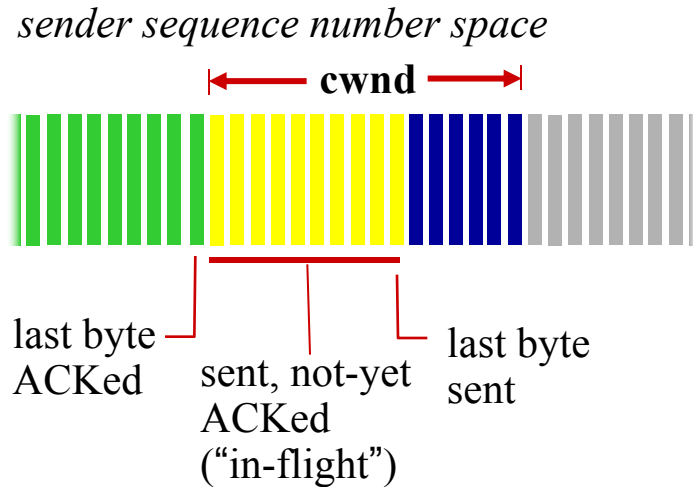
$\text{LastByteSent} - \text{LastByteAcked} \leq \min\{\text{cwnd}, \text{rwnd}\}$

Q2: How does a TCP sender perceive that there is congestion on the path between itself and the destination?

Timeout; three duplicate ACKs

Q3: What algorithm should the sender use to change its send rate as a function of perceived end-to-end congestion?

Congestion window



- ❖ sender limits transmission:

$$\text{LastByteSent} - \text{LastByteAcked} \leq \text{cwnd}$$

- ❖ **cwnd** is dynamic, function of perceived network congestion

TCP sending rate:

- ❖ *roughly*: send cwnd bytes, wait RTT for ACKS, then send more bytes

发送端最多能让 cwnd 字节处于“飞行中”

$$\text{rate} \approx \frac{\text{cwnd}}{\text{RTT}} \text{ bytes/sec}$$

这些字节要经过 大约 1 个 RTT 才会被 ACK/“回收额度”

一旦 ACK 回来，窗口释放出空间，发送端才能继续再发新的数据填满窗口

TCP congestion control: additive increase multiplicative decrease

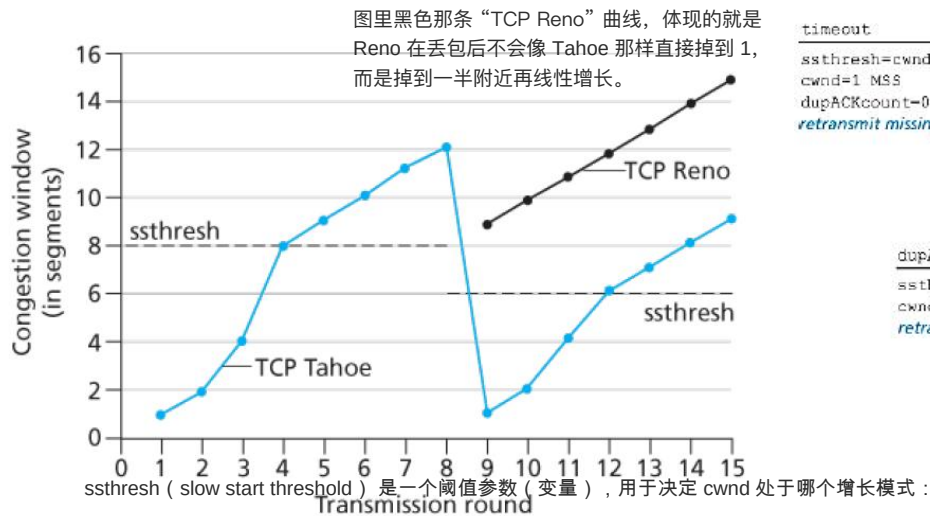
Q3: What algorithm should the sender use to **change its send rate** as a function of perceived end-to-end congestion?

- A lost segment → congestion → decrease rate
- An acknowledged segment → the network is fine → increase rate
- Bandwidth probing: network condition may change

TCP Congestion Control: details

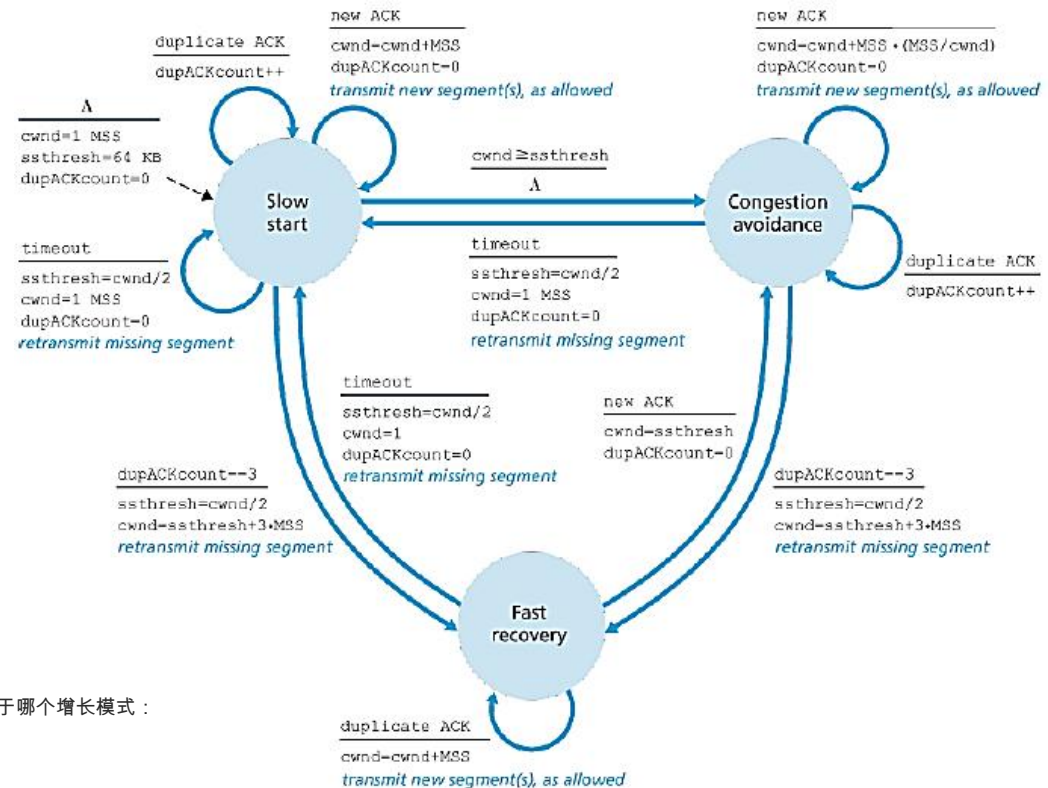
The congestion control algorithm has three major components:

- ❖ **Slow start**: exponentially increase
- ❖ **Congestion avoidance**: linearly increase
- ❖ **Fast recovery**:



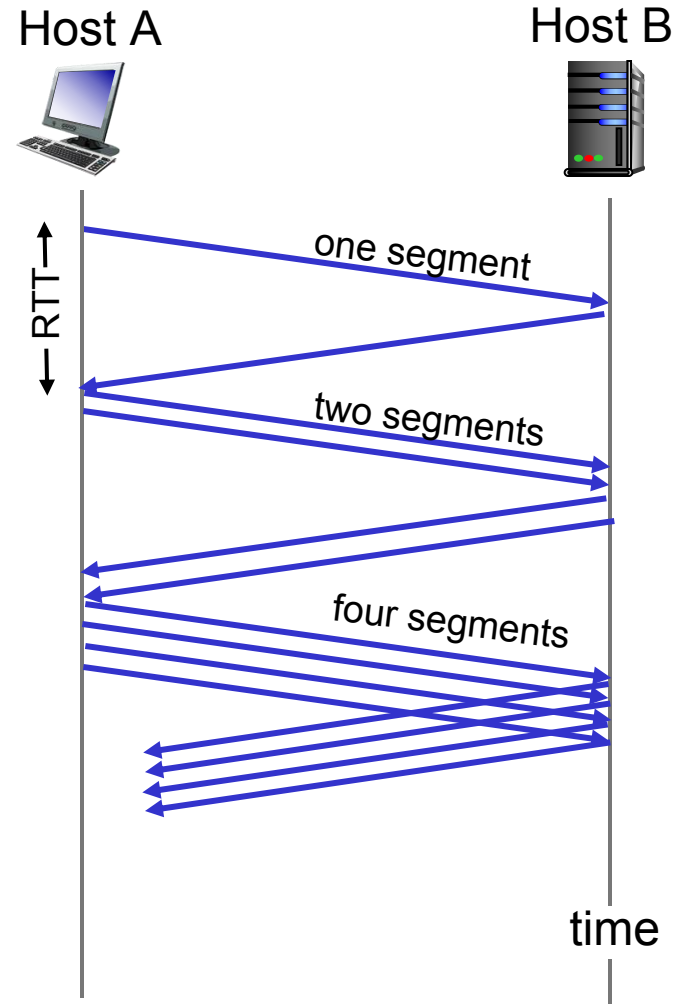
当 $cwnd < sssthresh$: 走 Slow Start (指数增长)

当 $cwnd \geq sssthresh$: 走 Congestion Avoidance (线性增长)

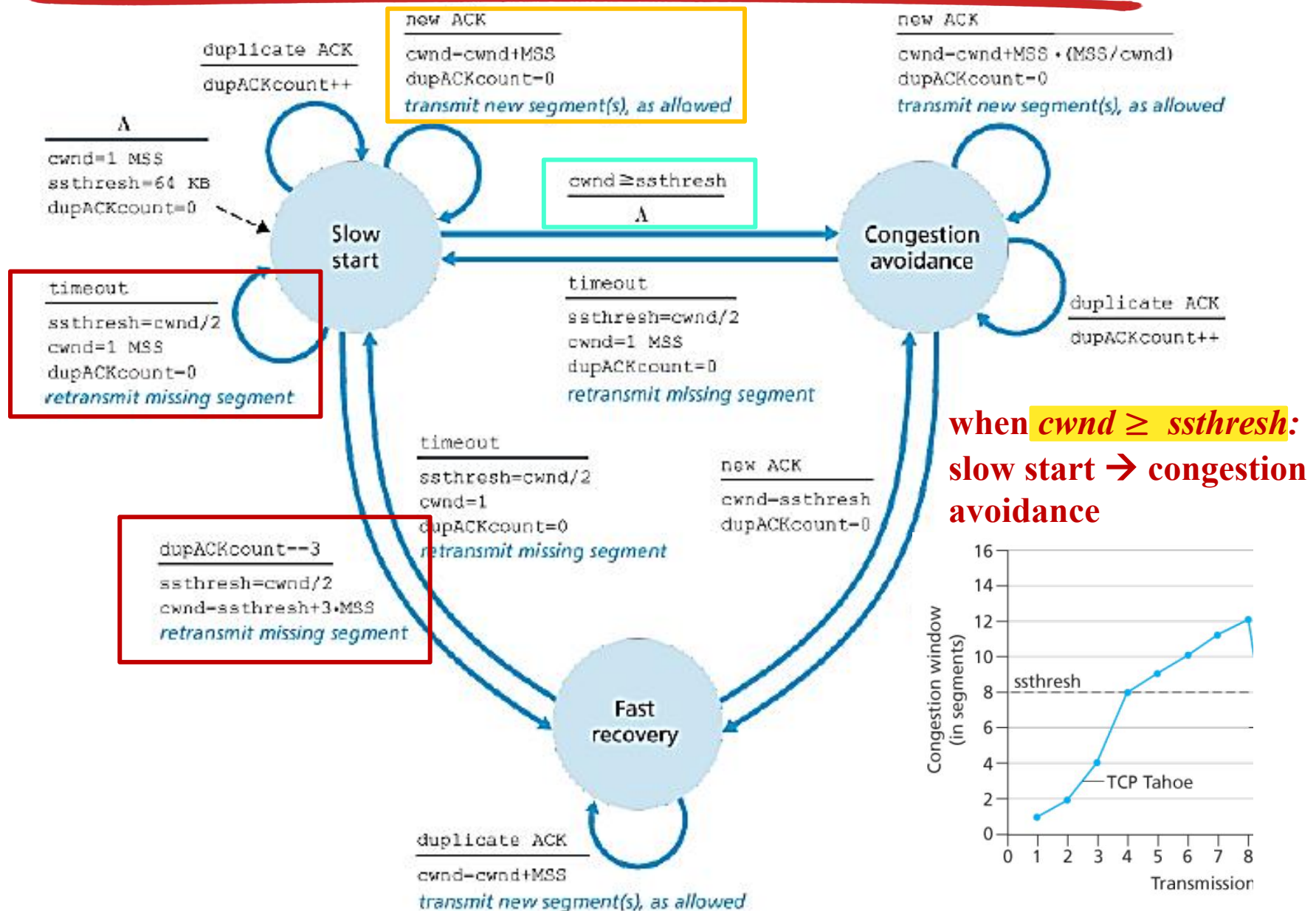


TCP Slow Start

- ❖ when **connection begins** or **timeout** occurs, increase rate **exponentially** until packet lost:
 - initially **cwnd** = 1 MSS
 - **double cwnd every RTT**
 - done by incrementing **cwnd** for every ACK received
- ❖ **Summary:** initial rate is slow but ramps up exponentially fast



TCP Congestion Control: FSM



TCP: detecting, reacting to loss

❖ loss indicated by timeout:

- **cwnd set to 1 MSS; ssthresh = cwnd/2**

TCP会将cwnd（拥塞窗口）重置为1个MSS（最大报文段大小），并将ssthresh（慢启动阈值）设置为当前cwnd的一半。

- window then grows exponentially (as in slow start) to threshold, then grows linearly

❖ loss indicated by 3 duplicate ACKs:

- TCP **Tahoe** always sets cwnd to 1 (timeout or 3 duplicate acks) → **Slow start**

- **cwnd set to 1 MSS; ssthresh = cwnd/2**

- TCP **RENO**

RENO认为: 3 dup
ACK ≠ 网络瘫痪

- dup ACKs indicate network capable of delivering some segments → **Fast Recovery**
- ssthresh = cwnd / 2; cwnd = ssthresh + 3MSS

TCP: switching from slow start to Congestion Avoidance

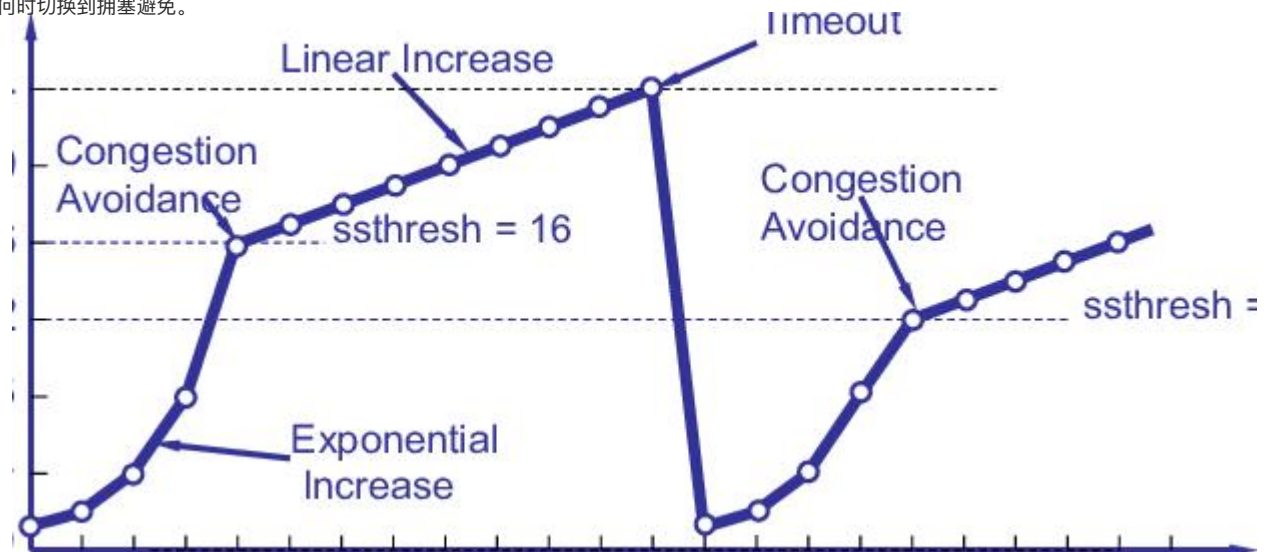
Q: when should the **exponential increase** switch to linear?

A: when **cwnd** gets to 1/2 of its value before timeout

Thus, when timeout occurs, $ssthresh = cwnd/2$

在发生丢包（超时）后，ssthresh被设置为原来cwnd的一半，并进入线性增长阶段

ssthresh 是由上一次拥塞事件确定的阈值，用于决定慢启动何时切换到拥塞避免。



TCP Congestion Avoidance

Trigger: $cwnd \geq ssthresh$

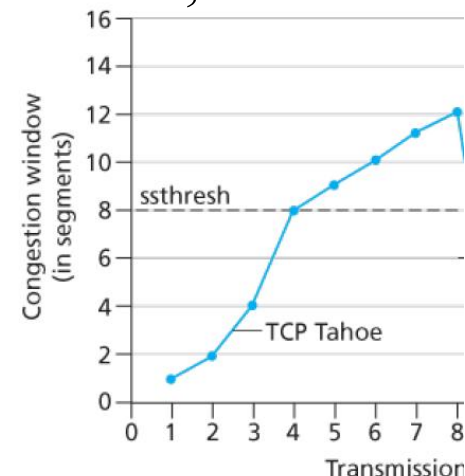
每个RTT增加一个MSS，而不是每个ACK增加一个MSS不然容易拥

Increases $cwnd$ **linearly**: by one MSS every RTT

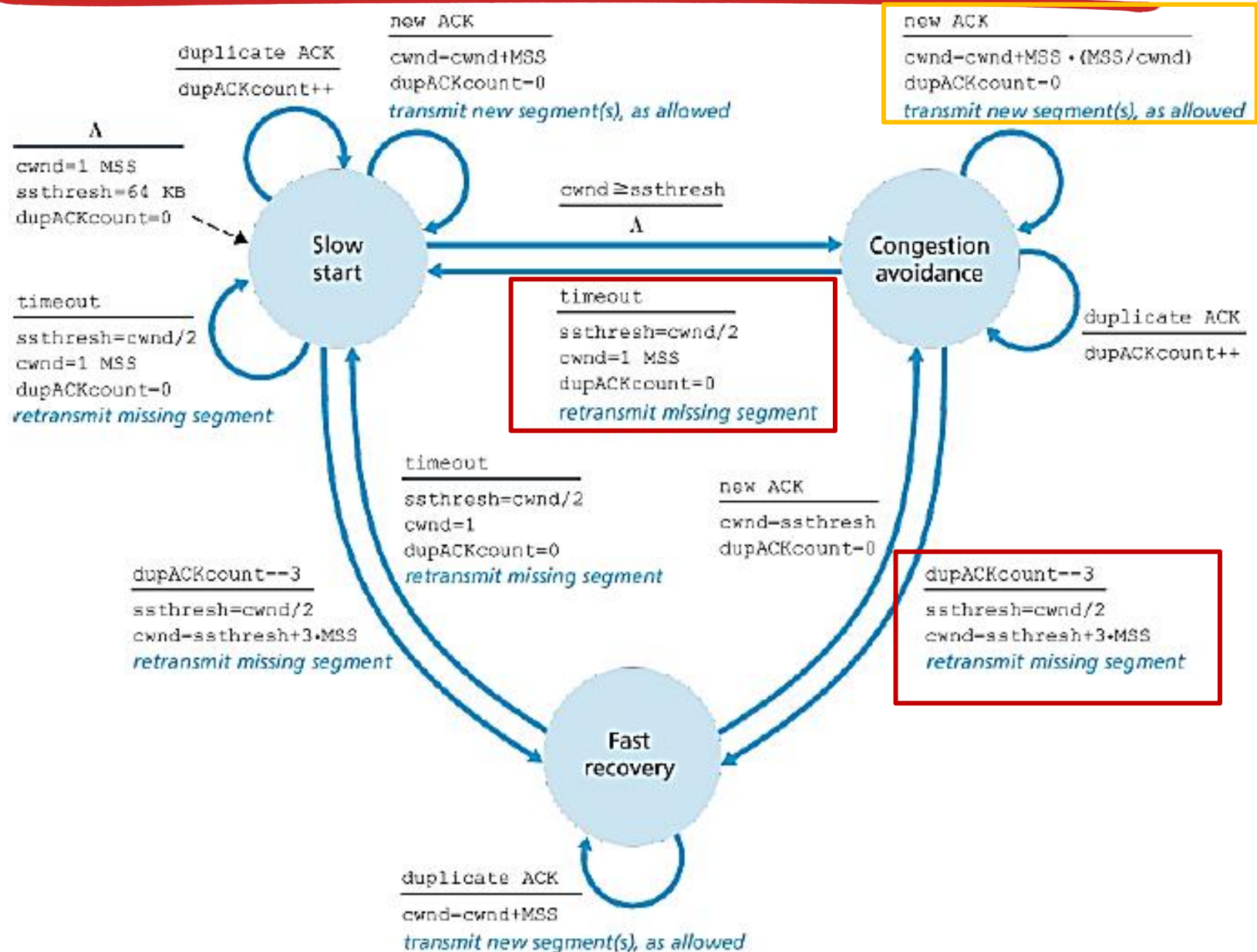
- ❖ Increase $cwnd$ by $(MSS/cwnd)MSS$ bytes whenever a new acknowledgment arrives.
- ❖ E.g., if MSS is 1,460 bytes and $cwnd$ is 14,600 bytes, then 10 segments are being sent within an RTT.
 - Each arriving ACK (assuming one ACK per segment) increases the congestion window size by 1/10 MSS,

“每来一个 ACK，就把 $cwnd$ 增加 $(MSS / cwnd) \cdot MSS$ ”，
却整体表现为：每个 RTT 增加 1 MSS（线性增长）

TCP 的“线性增长”不是靠时间触发的，而是靠 ACK 驱动实现的，所以应该认为是增加的MSS要均摊到每个ACK



TCP Congestion Control: FSM



TCP Fast Recovery

每来一个 dup ACK，说明又有一个“后续段”到达接收端

也就是说，网络里仍然有数据在流动，管道并没有空

把 $cwnd$ 设成 $ssthresh + 3 \times MSS$ ，相当于：

基础窗口砍半（到 $ssthresh$ ）

再“补偿”3 个段的 in-flight（对应那 3 个 dup ACK 反映的已达段）

这样发送端可以继续发新数据，不至于突然停住

直觉：砍半是为了降速，+3 是为了别让管道断流。

- ❖ Trigger (RENO) : triple duplicate ACKs
- ❖ $ssthresh = cwnd / 2; cwnd = ssthresh + 3MSS$

- ❖ The value of $cwnd$ is **increased by 1 MSS** for every duplicate ACK received for the **missing segment** that caused TCP to enter the fast-recovery state

Fast Recovery 期间：每来一个 dup ACK， $cwnd \pm 1$ MSS

- ❖ when an ACK arrives for the missing segment, enter congestion avoidance

当收到一个 新的 ACK，把那个缺口（丢的段）确认过去了（ACK 前移），说明：

丢的段已经被成功重传并接收

接收端不再卡在那个序号上

此时RENO： $cwnd = ssthresh$ ，进入congestion control 线性增长

假设丢包前：

- $cwnd = 12$ MSS

触发 3 dup ACK 后：

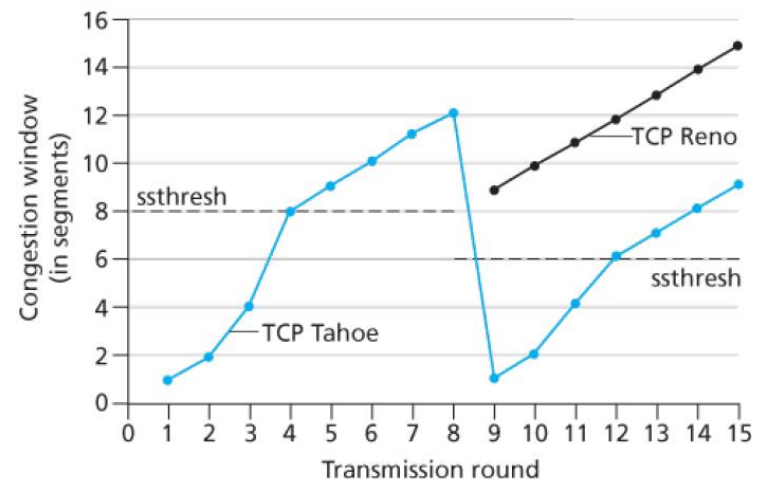
- $ssthresh = 6$ MSS
- $cwnd = 6 + 3 = 9$ MSS
- 立刻重传丢失段

接下来又来了 2 个 dup ACK：

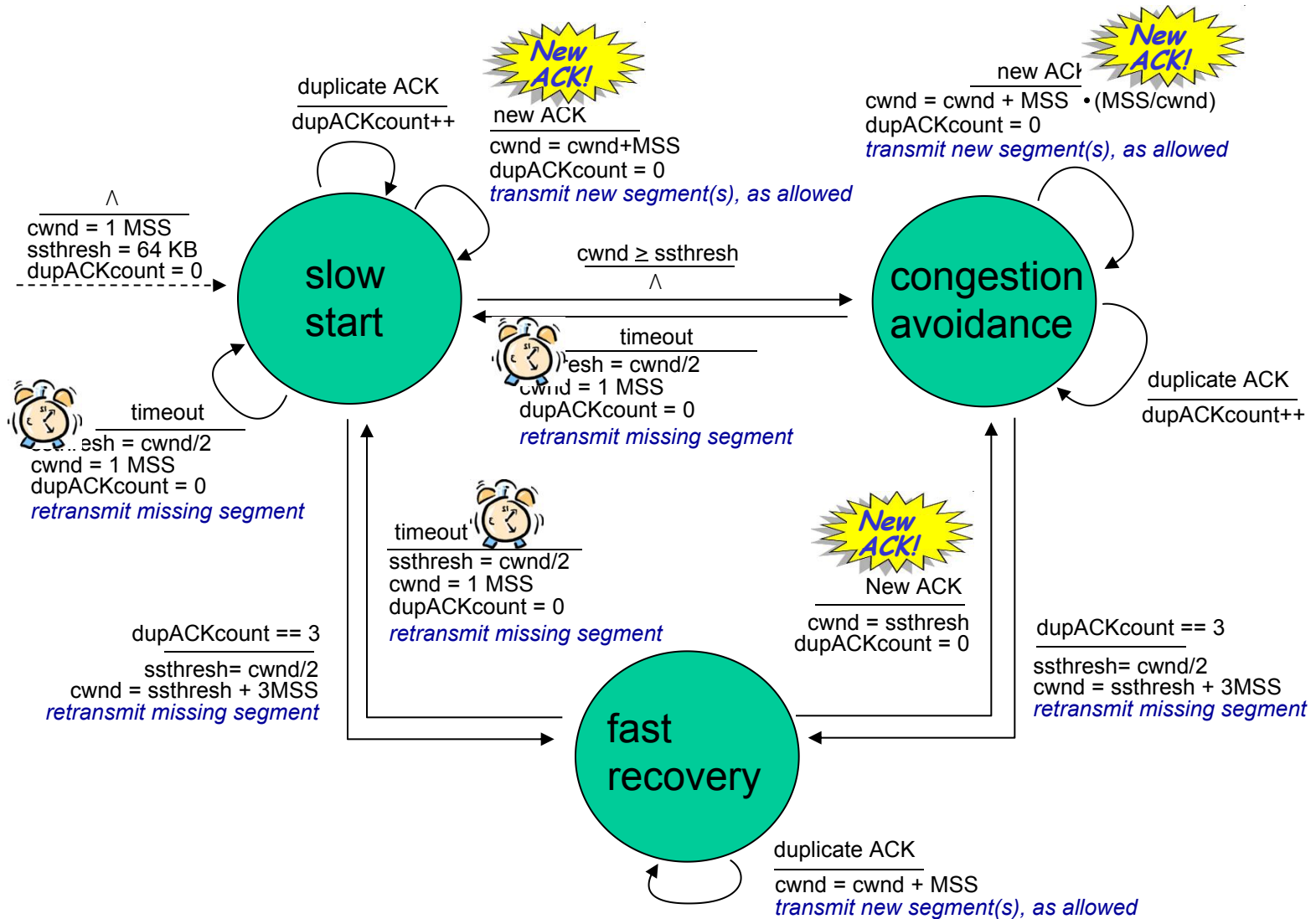
- $cwnd = 9 + 1 + 1 = 11$ MSS
- 每来一个 dup ACK 就允许再发一个新段（保持管道充满）

等到缺失段被确认（新 ACK 到来）：

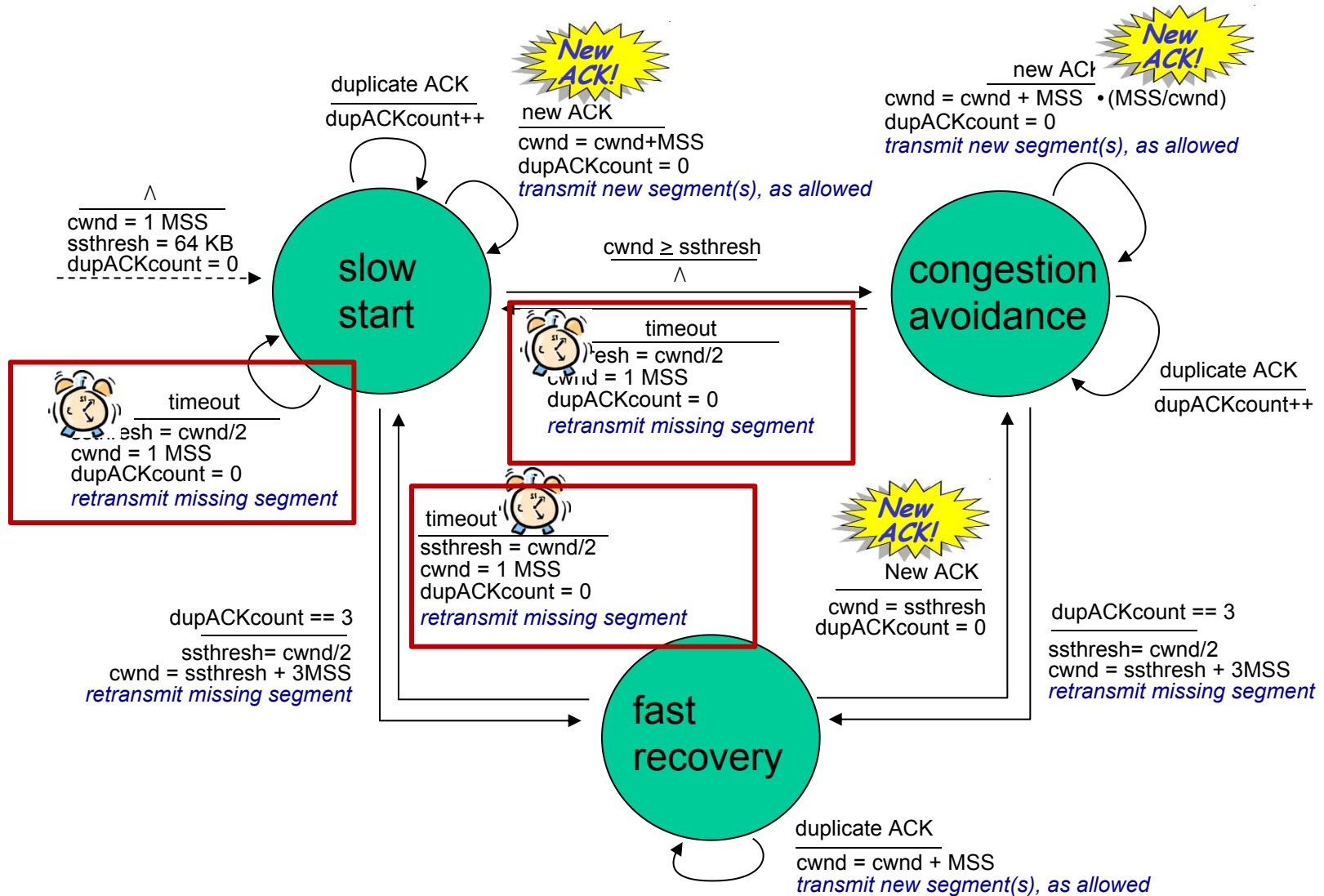
- $cwnd = ssthresh = 6$ MSS
- 转入拥塞避免开始线性加

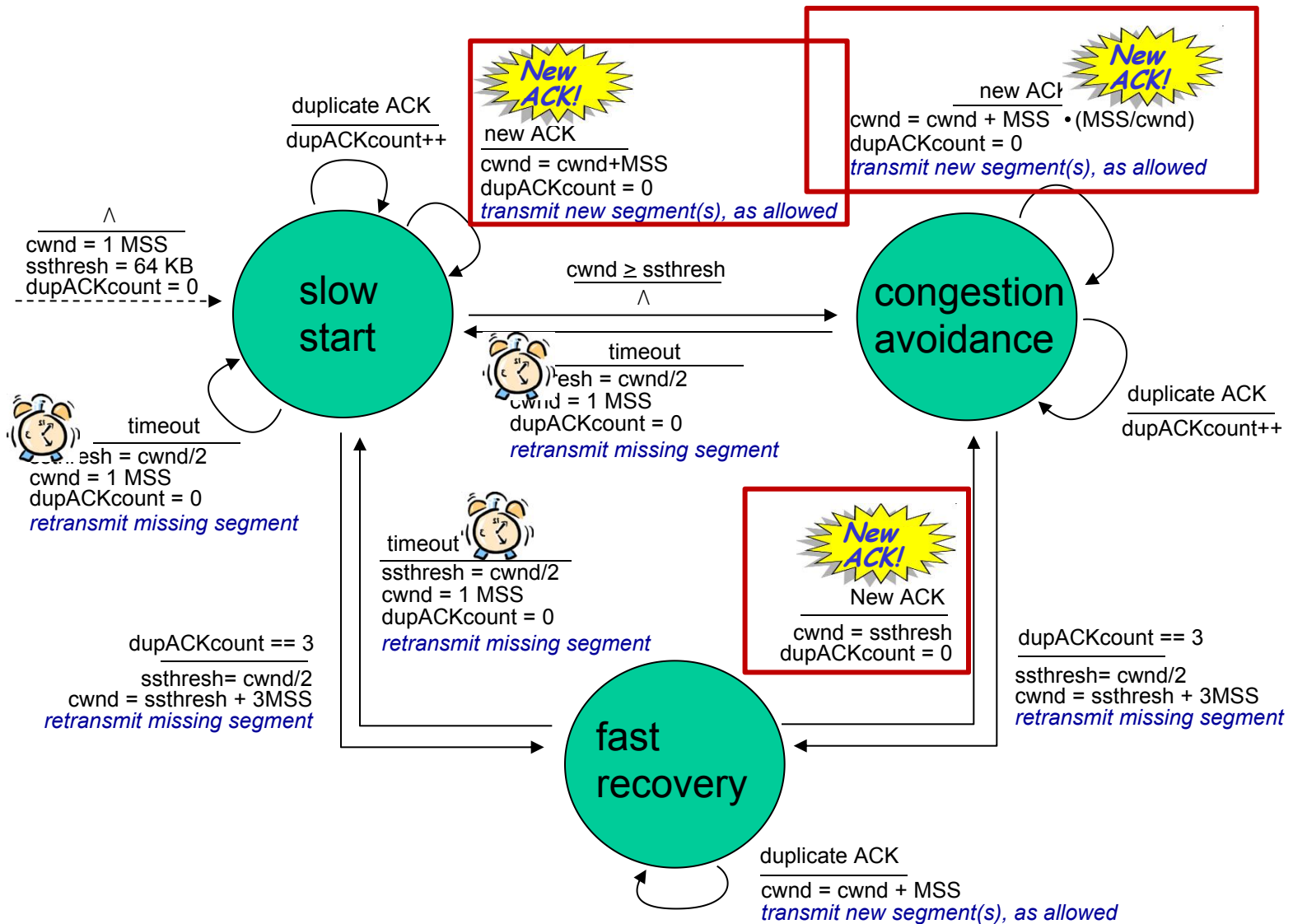


Summary: TCP Congestion Control

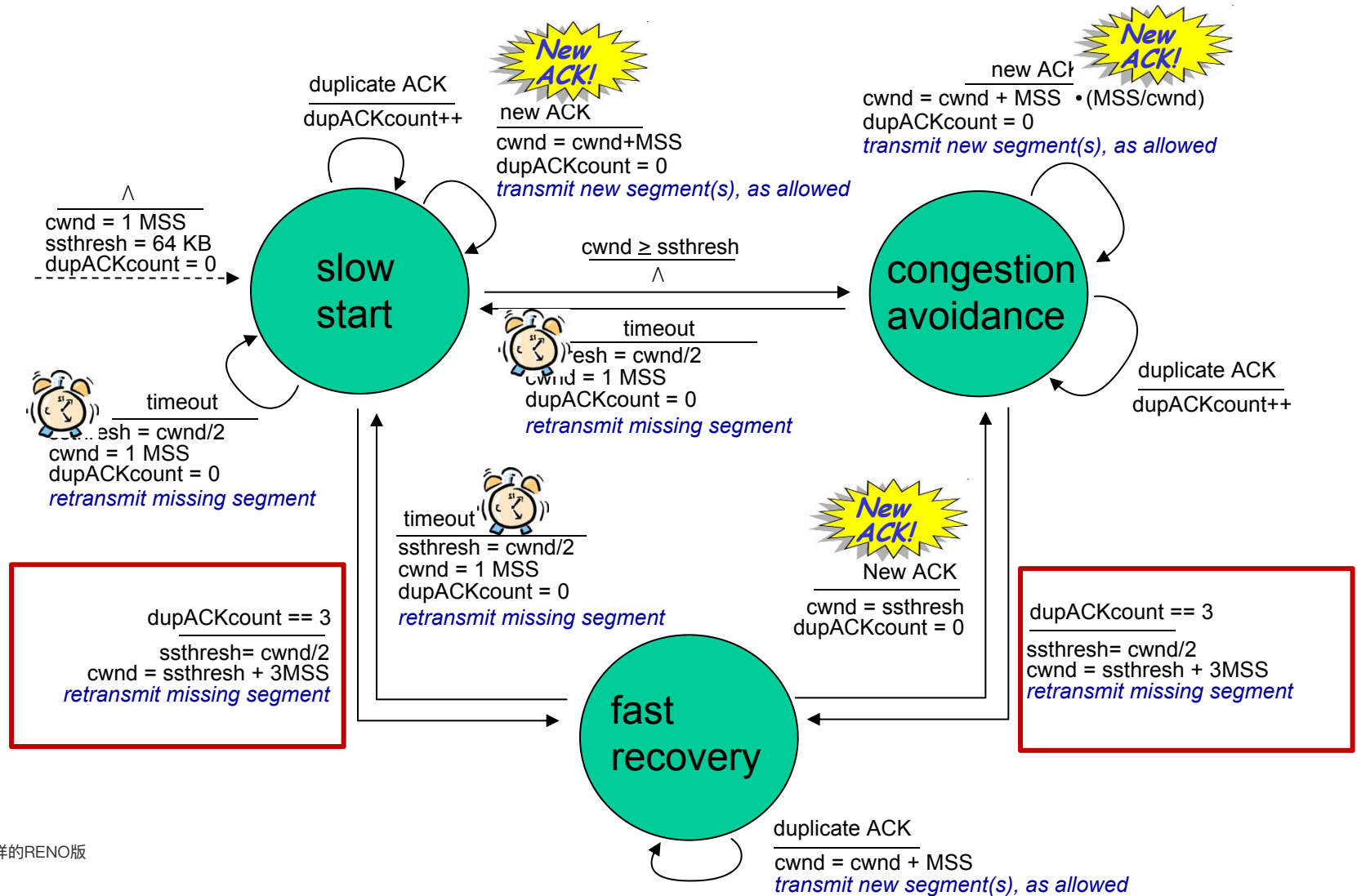


Summary: TCP Congestion Control





Summary: TCP Congestion Control



图中这样的RENO版

本:

timeout → slow start

3 dup ACK → fast

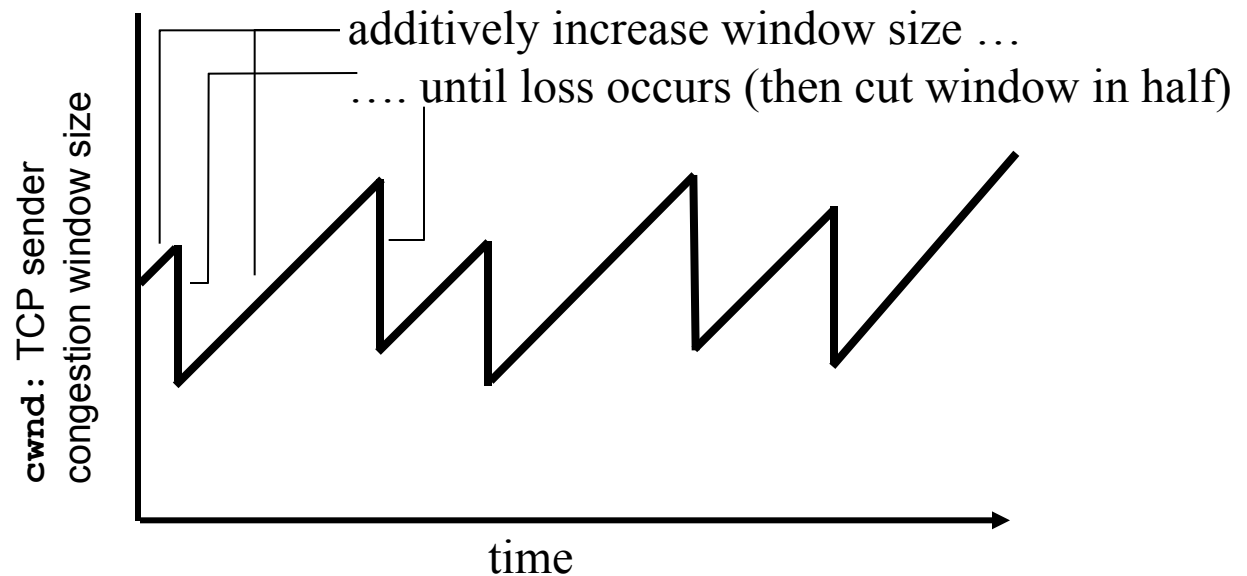
TCP congestion control: additive increase multiplicative decrease

AIMD

Approach: sender increases transmission rate (window size), **probing** for usable bandwidth, until loss occurs

- *additive increase:* increase **cwnd** by 1 MSS every RTT until loss detected
- *multiplicative decrease:* cut **cwnd** in half after loss

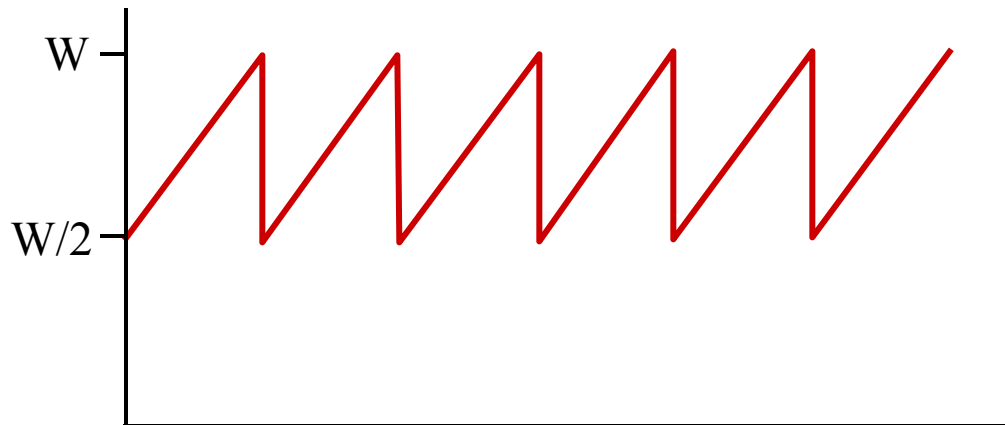
AIMD: probing for bandwidth



TCP throughput

- ❖ Avg. TCP throughput as function of window size, RTT?
 - ignore slow start, assume always data to send
- ❖ W: **window size** (measured in bytes) **where loss occurs**
 - avg. window size (# in-flight bytes) is $\frac{3}{4} W$
 - avg. throughput is $\frac{3}{4}W$ per RTT

$$\text{avg TCP thruput} = \frac{3}{4} \frac{W}{\text{RTT}} \text{ bytes/sec}$$



TCP Futures: TCP over “long, fat pipes”

$$\text{avg TCP thruput} = \frac{3}{4} \frac{W}{\text{RTT}} \text{bytes/sec}$$

每段1500 bytes

- ❖ example: 1500 byte segments, 100ms RTT, want 10 Gbps throughput 把 10 Gbps 换成 bytes/s
- ❖ requires $W = 83,333$ in-flight segments
- ❖ throughput **in terms of segment loss probability**, L [Mathis 1997]:

$$\text{TCP throughput} = \frac{1.22 \cdot \text{MSS}}{\text{RTT} \sqrt{L}}$$

→ to achieve 10 Gbps throughput, need a loss rate of $L = 2 \cdot 10^{-10}$ — *a very small loss rate!*

- ❖ new versions of TCP for high-speed

TCP Fairness

Fairness goal: if K TCP sessions share same bottleneck link of bandwidth R , each should have average rate of R/K

TCP connection 1



TCP connection 2



bottleneck
router
capacity R

TCP 为什么能“自然地”做到公平？
靠的就是 AIMD（加性增、乘性减）
情况分析（直觉版）：

两条连接都线性增加

- 谁快谁慢，加的“绝对值”一样
- 发生丢包时：
 - 窗口大的那条流，绝对减少更多
 - 因为是“乘性减小（砍一半）”

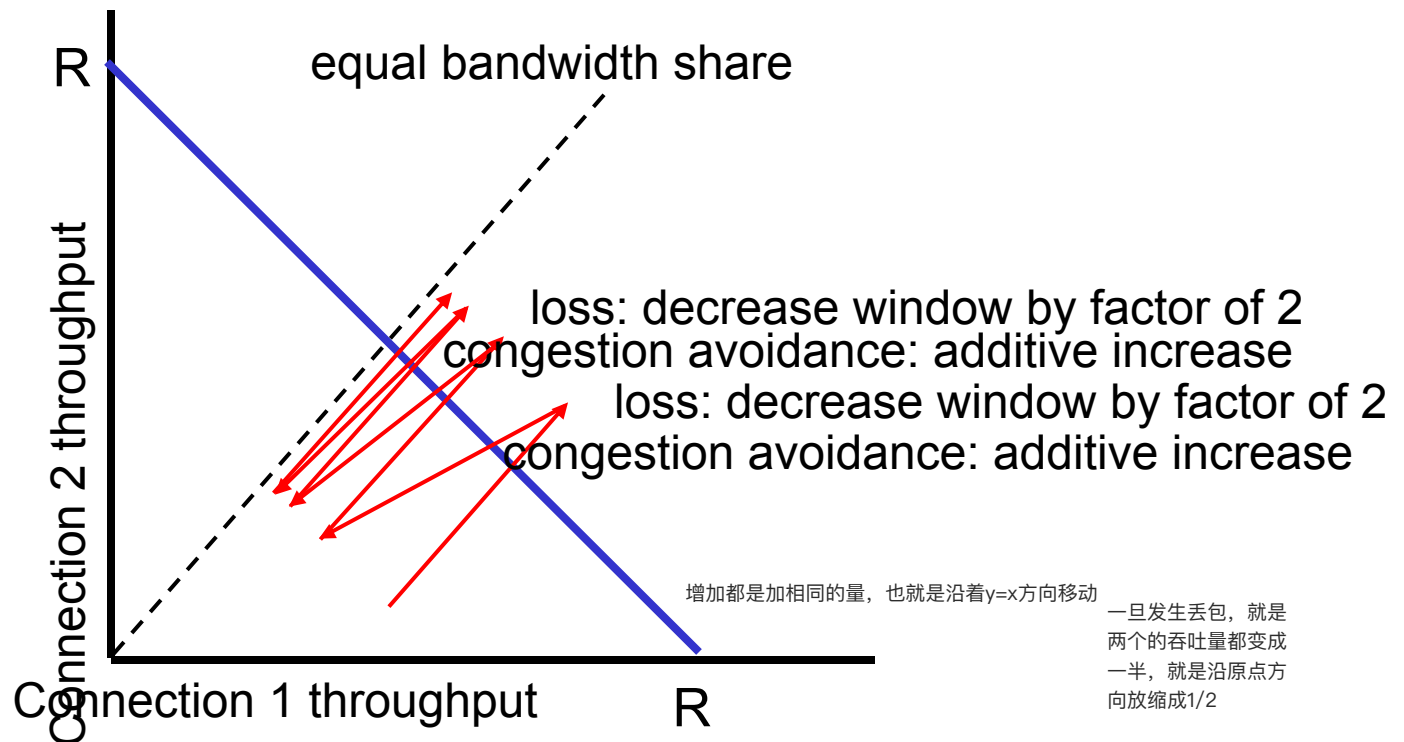
结果是：

- 大流降得多
- 小流降得少

Why is TCP fair?

two competing sessions:

- ❖ additive increase gives slope of 1, as throughput increases
- ❖ multiplicative decrease decreases throughput proportionally



Fairness (more)

Fairness and UDP

- ❖ multimedia apps often do not use TCP
 - do not want rate throttled by congestion control 不需要节流
- ❖ use UDP:
 - send audio/video at constant rate, tolerate packet loss
- ❖ UDP sources to crowd out TCP traffic
UDP 流量会把 TCP 挤出去

Fairness, parallel TCP connections

- ❖ application can open multiple parallel connections between two hosts
- ❖ web browsers do this
- ❖ e.g., link of rate R with 9 existing connections:
 - new app asks for 1 TCP, gets rate $R/10$
 - new app asks for 11 TCPs, gets more than $R/2$

TCP对TCP连接是公平的，但是对应用不一定是公平的

为什么浏览器要开多连接？

网页加载用多个 TCP 连接

加快单个应用的“体感速度”

本质上是 在公平规则允许的范围内抢资源

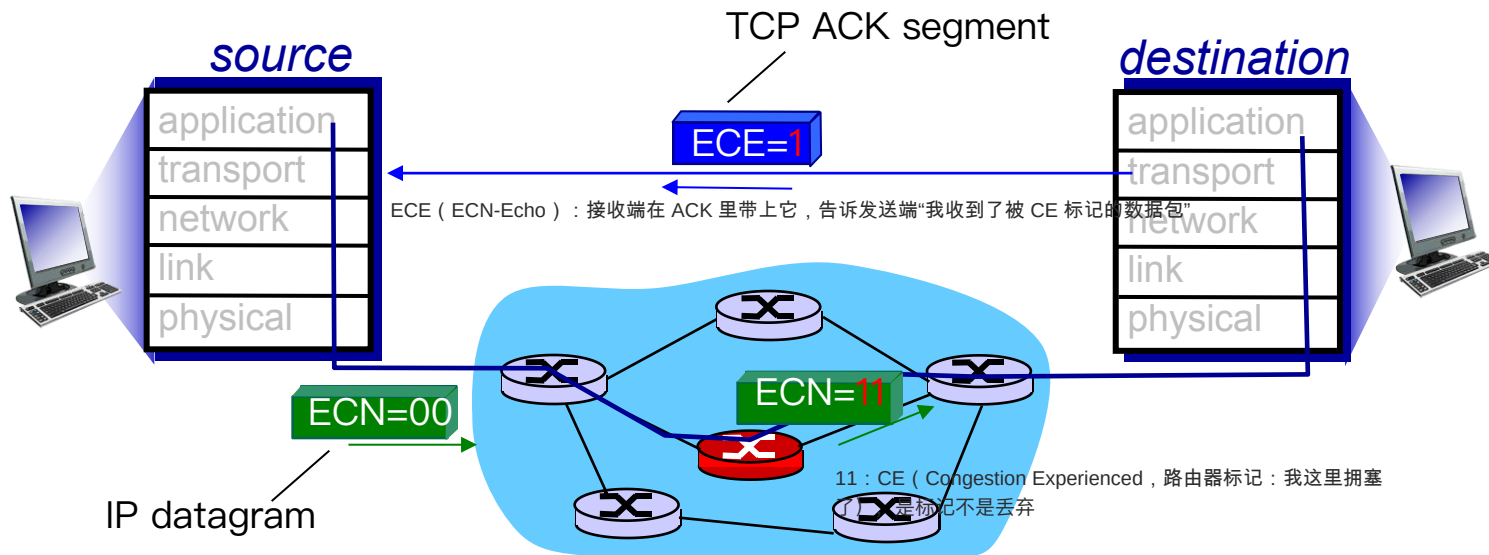
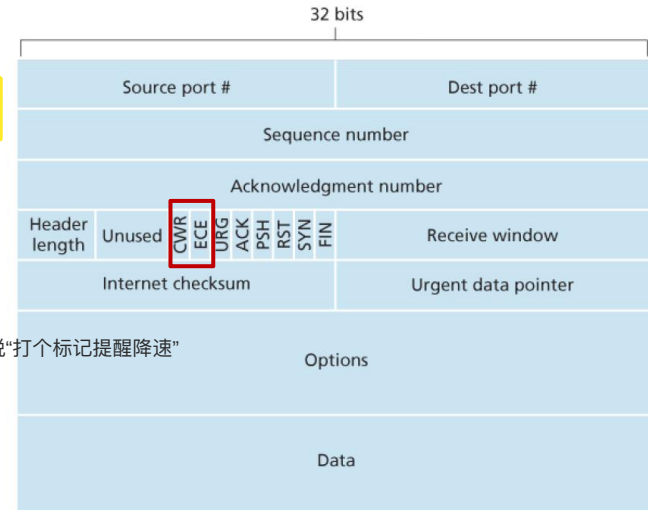
Explicit Congestion Notification (ECN)

network-assisted congestion control:

- two bits in IP header (ToS field) marked by network router to indicate congestion
- congestion indication carried to receiving host
- receiver sets ECE bit on receiver-to-sender ACK segment to notify sender of congestion

传统TCP靠timeout/dup ACK推测堵塞

ECN让路由器在队列开始变长、快拥塞时就说“打个标记提醒降速”



Chapter 3: summary

- ❖ principles behind transport layer services:
 - multiplexing, demultiplexing
 - reliable data transfer
 - flow control
 - congestion control
 - ❖ instantiation, implementation in the Internet
 - UDP
 - TCP
- next:
- ❖ leaving the network “edge” (application, transport layers)
 - ❖ into the network “core”